Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

Caso de Estudio Integral: Enterprise Architecture · Data Architecture · Agentic AI Architecture

Autor: Martín Galán
Proyecto: Agentic AI Bootcamp - Caso de Estudio 3
Repositorio: github.com/mgalanme/guardian-health
Versión: 1.0.0 - Draft Fundacional
Fecha: Abril 2026

Índice

1. Introducción **3**
1.1 Qué construiremos 3
1.2 Por qué este Caso de Uso 3
1.3 Tecnologías, Frameworks y Justificación Arquitectónica 4
Stack de Orquestación Agéntica 4
Infraestructura de Datos 5
Capa de Integración y APIs 5
Inferencia y Observabilidad 6
1.4 El Principio Central: Gobierno desde el Día Cero 6
2. Caso de Estudio: GUARDIAN-Health **7**
2.1 Contexto y Problema 7
2.2 Descripción del Sistema 8
2.3 Stakeholders y Actores 9
2.4 Requisitos Funcionales y No Funcionales 10
2.5 Restricciones Regulatorias y de Cumplimiento 11
3. Arquitectura Empresarial **12**
3.1 Fase Preliminar: Establecimiento del Marco 12
3.2 Fase A: Visión de Arquitectura 13
3.3 Fase B: Arquitectura de Negocio 14
3.4 Fase C: Arquitectura de Sistemas de Información 15
3.5 Fase D: Arquitectura Tecnológica 16
3.6 Gobierno de Arquitectura y ADRs 16
4. Arquitectura de Datos **17**
4.1 Gobierno del Dato: Estructura y Responsabilidades 17
4.2 Arquitectura de Datos y Modelo Conceptual 19
4.3 Gestión de Datos Maestros y de Referencia 20
4.4 Calidad del Dato en Contextos Agénticos 20
4.5 Linaje, Metadatos y Trazabilidad 22

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

4.6 Pipeline de Datos: Arquitectura Medallion en Salud 22
4.7 Privacidad, Seguridad y Cumplimiento del Dato 23
5. Arquitectura de IA Agéntica 24
5.1 Patrones de Agente y Justificación de Selección 24
5.2 Topología Multi-Agente: Roles y Responsabilidades 25
5.3 Orquestación con LangGraph 25
5.4 Crews con CrewAI y AutoGen 28
5.5 Diseño de Memoria: Corto, Largo Plazo y Episódica 29
5.6 Human-in-the-Loop (HITL): Diseño y Justificación 30
5.7 Herramientas y APIs Externas 31
5.8 Mensajería Orientada a Eventos: Solace Agent Mesh 32
5.9 Observabilidad Agéntica: LangSmith + Trazas de Agente 33
6. Gobierno y Responsabilidad por Diseño 34
6.1 Por qué el Gobierno ha de ser desde el Día Cero 34
6.2 Riesgos de Ausencia de Gobierno Agéntico Temprano 35
6.3 Framework de Gobierno para Sistemas Agénticos 36
6.4 Trazabilidad de Decisiones Agénticas (Audit Trail) 37
6.5 Explicabilidad y Transparencia 38
7. Plan de Implementación 39
7.1 Entorno de Desarrollo y Contenedores 39
7.2 Estructura del Repositorio 40
7.3 Roadmap de Desarrollo por Fases 41
8. Apéndices 42
A. Registro de Decisiones de Arquitectura (ADRs) 42
B. Glosario 44
C. Referencias 45

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

1. Introducción

1.1 Qué construiremos

GUARDIAN-Health es una POC de un sistema de IA agéntica para la monitorización continua de la seguridad del paciente en una red hospitalaria multi-centro. El sistema detecta, clasifica y gestiona eventos adversos clínicos y señales de farmacovigilancia, coordinando respuestas entre departamentos clínicos, farmacia, dirección médica y autoridades regulatorias.

El propósito arquitectónico de este caso de estudio trasciende la funcionalidad: es una demostración ejecutable de que el **Gobierno, la Trazabilidad y la Responsabilidad** en sistemas de IA agéntica no pueden ser añadidos a posteriori. Deben estar tejidos en el diseño desde la primera línea de código, la primera decisión de datos y el primer nodo del grafo.

El sistema implementará 3 módulos agénticos interconectados:

Módulo 1 - VIGIL (Vigilancia y Detección): Agentes que monitorizan en tiempo real señales clínicas, prescripciones, resultados de laboratorio y registros de enfermería para identificar patrones anómalos potencialmente indicativos de eventos adversos.

Módulo 2 - ASSESS (Evaluación y Clasificación): Crew de agentes que evalúa la gravedad, causalidad y notificabilidad de cada señal detectada, produciendo recomendaciones estructuradas con trazabilidad de razonamiento completa.

Módulo 3 - RESPOND (Coordinación de Respuesta): Agentes de coordinación que gestionan la comunicación entre actores clínicos, escalan según protocolos HITL definidos, y generan la documentación regulatoria requerida (notificaciones a la AEMPS, informes de seguridad).

1.2 Por qué este Caso de Uso

La farmacovigilancia y la seguridad del paciente en entornos hospitalarios representan uno de los dominios más exigentes para sistemas de IA agéntica autónoma. Esta elección es deliberada y arquitectónicamente motivada:

- **Consecuencias críticas de las decisiones:** Un agente que clasifica erróneamente un evento adverso grave como leve puede retrasar intervenciones que salvan vidas. Esto hace que el gobierno de las decisiones agénticas no sea opcional, sino una restricción dura de diseño.

- **Densidad regulatoria:** GDPR, la Ley Orgánica de Protección de Datos (LOPDGDD), la normativa de farmacovigilancia europea (Regulación (UE) No 1235/2010), y las directrices de la Agencia Europea del Medicamento (EMA) crean un entorno donde cada acción del agente debe ser justificable, trazable y auditable.

- **Heterogeneidad de datos:** Los datos clínicos combinan registros estructurados (HL7 FHIR, bases de prescripción), semiestructurados (notas clínicas, informes de alta) y no estructurados (narrativas de enfermería, comunicaciones internas). Esto requiere el uso coordinado de bases de datos relacionales, grafos y vectoriales.

- **Múltiples partes interesadas con intereses divergentes:** Clínicos, administración hospitalaria, autoridades regulatorias, pacientes y aseguradoras tienen perspectivas y niveles de confianza distintos sobre las decisiones del sistema, lo que exige un diseño de gobierno que pueda satisfacer múltiples marcos de rendición de cuentas simultáneamente.

- **Valor real demostrable:** La farmacovigilancia activa basada en IA está identificada como una prioridad estratégica tanto por la EMA como por la AEMPS. Un caso de estudio en este dominio tiene relevancia directa para proyectos reales en el sector sanitario español y europeo.

1.3 Tecnologías, Frameworks y Justificación Arquitectónica

La selección tecnológica no es arbitraria. Cada componente responde a un requisito arquitectónico específico de gobierno, trazabilidad, o capacidad agéntica.

Stack de Orquestación Agéntica

- **LangChain** actúa como la capa de abstracción fundamental para la integración de LLMs, herramientas y cadenas de procesamiento. Proporciona los conectores a Groq, las abstracciones de vector stores y el sistema de prompts estructurados con soporte nativo para trazabilidad. Su elección se justifica por su madurez en producción y la amplia ecosistema de integraciones, crítico cuando el sistema necesita conectar con múltiples fuentes de datos clínicos heterogéneas.

- **LangGraph** es el orquestador de agentes con estado. En un sistema de farmacovigilancia, el estado es central: un evento adverso pasa por múltiples estados (detectado, en evaluación, clasificado, notificado, cerrado) con transiciones que deben ser explícitas, auditables y reversibles bajo supervisión humana. LangGraph implementa este flujo como un grafo dirigido con gestión de estado persistente, soporte nativo para HITL mediante interrupciones de nodo, y capacidad de checkpointing que permite retroceder en el flujo de decisión para auditoría.

- **CrewAI** gestiona los crews de evaluación en el Módulo ASSESS. Cuando múltiples agentes especializados (farmacólogo, clínico, experto en regulación) deben colaborar en la evaluación de una señal, CrewAI proporciona la abstracción de roles, la delegación estructurada y el proceso de consenso. Su modelo de tareas con salidas tipadas en Pydantic garantiza que cada paso del razonamiento colectivo produzca artefactos estructurados y trazables.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

- **AutoGen** se emplea para escenarios de evaluación adversarial y simulación de protocolos. La capacidad de AutoGen para crear conversaciones multi-agente con roles de usuario y asistente definidos formalmente es especialmente útil para simular la discusión entre un agente proponente de clasificación y un agente revisor crítico, replicando el proceso de revisión por pares en evaluación de seguridad.

Infraestructura de Datos

- **Neo4j** es la base de datos de grafos para el mapa de relaciones clínicas: pacientes, medicamentos, prescriptores, diagnósticos, eventos adversos y protocolos de tratamiento. Las señales de farmacovigilancia son inherentemente relacionales: detectar que un medicamento causa una reacción adversa en pacientes con una combinación específica de comorbilidades requiere recorrer grafos de relaciones, no consultar tablas. Los algoritmos de detección de patrones en Neo4j (mediante Cypher y APOC) permiten identificar señales que serían invisibles en modelos relacionales planos.

- **PostgreSQL** es la base de datos relacional para datos transaccionales, registros de auditoría, estado del flujo agéntico (mediante el checkpointer de LangGraph) y datos maestros. Su elección sobre alternativas NoSQL se justifica por su soporte de transacciones ACID, crítico cuando se registran decisiones regulatorias con implicaciones legales.

- **Pinecone** proporciona el vector store en la nube para las búsquedas semánticas sobre literatura médica, guías clínicas, fichas técnicas de medicamentos y protocolos hospitalarios. A diferencia de Qdrant (que también usamos localmente para desarrollo), Pinecone en producción ofrece escalabilidad gestionada y SLAs adecuados para entornos clínicos.

- **Qdrant** se emplea en el entorno de desarrollo local como alternativa a Pinecone, manteniendo la misma interfaz de abstracción de LangChain para garantizar portabilidad entre entornos.

Capa de Integración y APIs

- **FastAPI** expone los servicios del sistema como APIs REST con documentación OpenAPI automática. En un entorno hospitalario, la interoperabilidad es no negociable: los sistemas de Historia Clínica Electrónica (HCE), los sistemas de prescripción y los portales de reportes regulatorios deben poder integrarse con GUARDIAN-Health mediante contratos de API bien definidos y versionados. FastAPI con Pydantic garantiza la validación estricta de tipos en los límites del sistema.

- **Solace Agent Mesh** implementa la mensajería orientada a eventos para la coordinación asíncrona entre módulos. En un sistema hospitalario real, los eventos (una nueva prescripción, un resultado de laboratorio anómalo, una alerta de enfermería) llegan de forma asíncrona desde múltiples sistemas fuente. Solace proporciona un bus de eventos con garantías de entrega, soporte para patrones publish-subscribe y la capacidad de integrar el sistema agéntico con el ecosistema de mensajería hospitalaria existente (HL7 v2, FHIR R4).

- **n8n** (auto-hospedado, tier gratuito) actúa como motor de workflow para automatizaciones no-agénticas: notificaciones programadas, generación de informes periódicos, sincronización de datos de referencia y triggers de procesos regulatorios rutinarios. Su interfaz visual permite que personal no técnico del equipo de farmacovigilancia configure y monitorice workflows sin necesidad de modificar código.

Inferencia y Observabilidad

- **Groq** proporciona inferencia de LLM de ultra-baja latencia con `llama-3.3-70b-versatile` como modelo principal. En un contexto clínico, la latencia importa: un sistema de alertas de seguridad que tarda minutos en procesar una señal no es útil en urgencias. Groq permite procesar señales complejas en segundos con la capacidad de razonamiento de modelos de 70B parámetros.

- **LangSmith** es la capa de observabilidad para todas las trazas agénticas. Cada invocación de LLM, cada ejecución de herramienta, cada transición de estado y cada decisión HITL queda registrada en LangSmith con su contexto completo. En un sistema regulado, esto no es una herramienta de debugging, es una pieza del sistema de gobierno: la traza de LangSmith es parte del audit trail.

- **HuggingFace** (`nomic-ai/nomic-embed-text-v1`, 768 dims) provee los embeddings para la indexación semántica de documentos clínicos. La elección de un modelo de embeddings de código abierto garantiza reproducibilidad y auditoría del proceso de vectorización.

1.4 El Principio Central: Gobierno desde el Día Cero

Este caso de estudio existe para demostrar un principio que la industria aprende, invariablemente, de la manera difícil: **en sistemas de IA agéntica, el gobierno retroactivo no existe**.

Un sistema agéntico que empieza sin gobierno desarrolla, en cuestión de semanas, una deuda de gobierno imposible de saldar:

- Las decisiones del agente se vuelven opacas porque nadie diseñó la estructura de audit trail desde el principio.
- Los datos de entrenamiento y contexto no tienen lineage porque nadie estableció los metadatos desde el origen.
- Las responsabilidades son ambiguas porque los roles de supervisión humana no se definieron como parte del flujo agéntico.
- El cumplimiento regulatorio se vuelve una pesadilla forense porque los registros no tienen la estructura que requieren los auditores.

En los capítulos que siguen, cada decisión de diseño arquitectónico está motivada por un requisito de gobierno. El gobierno no es una capa que se añade al final. Es la columna vertebral del sistema.

2. Caso de Estudio: GUARDIAN-Health

2.1 Contexto y Problema

Red HospitalARIA es una red hospitalaria ficticia pero arquitectónicamente realista, compuesta por 3 hospitales de nivel terciario en España (Madrid, Barcelona y Sevilla), con un total de 2.400 camas, 180.000 pacientes atendidos al año y un gasto farmacéutico anual de 45 millones de euros.

La red tiene un Departamento Central de Farmacovigilancia (DCF) que, conforme al Real Decreto 577/2013 y la normativa europea, es responsable de:

- Detectar y notificar sospechas de reacciones adversas a medicamentos (RAM) a la AEMPS.
- Mantener el Sistema de Gestión de Riesgos de cada medicamento en uso.
- Evaluar las señales de seguridad emergentes.
- Coordinar con la Comisión de Farmacia y Terapéutica de cada centro.

El problema: Con 2.400 camas activas y una media de 8,2 medicamentos por paciente ingresado, el DCF gestiona potencialmente 19.680 combinaciones de medicamento-paciente simultáneas. El equipo de farmacovigilancia tiene 12 personas. La detección de señales es reactiva: se basa en notificaciones voluntarias de clínicos (con una tasa de infranotificación estimada del 94% según datos de la EMA) y en revisiones periódicas de historiales que tardan semanas en completarse.

El resultado medible del problema: durante el período 2022-2024, la red detectó 234 eventos adversos graves. Un análisis retrospectivo mostró que 67 de ellos (28,6%) presentaban señales detectables en los datos clínicos con una antelación de 24-72 horas antes de la manifestación del evento. El sistema actual no tiene capacidad de detección prospectiva.

La oportunidad: Un sistema de IA agéntica con monitorización continua puede transformar el proceso de farmacovigilancia de reactivo a proactivo, pasando de detectar eventos a prevenir sus consecuencias. Esta transformación, sin embargo, requiere un nivel de gobierno que justifique que las recomendaciones del sistema son fiables, explicables y auditables antes de que cualquier clínico actúe sobre ellas.

2.2 Descripción del Sistema

GUARDIAN-Health implementa un flujo agéntico continuo organizado en tres módulos que operan de forma coordinada:

Módulo VIGIL ejecuta agentes de monitorización que se conectan, mediante herramientas especializadas, a:

- El sistema de prescripción electrónica (simulado mediante datos FHIR sintéticos)
- Los resultados del sistema de laboratorio (valores fuera de rango, cambios significativos)
- Los registros de signos vitales de enfermería
- Las notas de evolución médica (procesadas mediante NLP con embeddings semánticos)
- Las alertas del sistema de interacciones farmacológicas.

Estos agentes producen **señales brutas**: observaciones estructuradas que indican que algo requiere atención, sin emitir juicio clínico sobre su significado.

Módulo ASSESS recibe las señales brutas mediante el bus de eventos (Solace) y activa un crew de evaluación que incluye:

- Un agente **Farmacólogo** que evalúa la plausibilidad biológica de la relación causal
- Un agente **Clínico** que evalúa el contexto del paciente y las alternativas diagnósticas
- Un agente **Regulatorio** que determina si el evento es notificable según la normativa vigente
- Un agente **Síntesis** que produce la evaluación final estructurada.

Cada agente documenta su razonamiento en formato estructurado. La evaluación final incluye nivel de confianza, evidencia de soporte, y grado de consenso entre agentes.

Módulo RESPOND gestiona las acciones derivadas de las evaluaciones:

- Notificaciones a clínicos responsables (con urgencia calibrada al nivel de riesgo)
- Escalado a supervisión humana para eventos de alta gravedad (HITL obligatorio)
- Preparación de reportes de notificación a la AEMPS
- Actualización del grafo de relaciones en Neo4j
- Registro completo del evento y su gestión en el audit trail.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

2.3 Stakeholders y Actores

Actor	Rol	Interacción con el sistema	Nivel de confianza requerido
Médico prescriptor	Recibe alertas, confirma o descarta señales	Dashboard clínico / notificación	Debe poder explicar el razonamiento del agente
Farmacólogo clínico	Revisa evaluaciones del Módulo ASSESS	Portal de farmacovigilancia	Necesita trazabilidad completa de la evaluación
Director médico	Aprueba notificaciones regulatorias	Flujo HITL obligatorio	Firma digital sobre decisiones de notificación
DCF (Farmacovigilancia)	Gestiona el sistema, define protocolos	Consola de administración	Control total de parámetros y thresholds
AEMPS	Recibe notificaciones regulatorias	API de notificación (E2B R3)	Conformidad regulatoria del formato y contenido
Paciente	Sujeto de los datos	Sin interacción directa	Protección de datos y no discriminación
Comité de Ética	Supervisión del sistema	Informes de gobierno	Acceso a métricas de equidad y sesgo
Auditores de datos	Verificación de cumplimiento	Audit trail completo	Trazabilidad de cada decisión agéntica

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

2.4 Requisitos Funcionales y No Funcionales

Requisitos Funcionales:

- RF-01: El sistema debe monitorizar continuamente las señales clínicas de todos los pacientes ingresados con una latencia máxima de 15 minutos desde la generación del dato.

- RF-02: El Módulo ASSESS debe producir evaluaciones estructuradas con nivel de confianza, evidencia de soporte y razonamiento explícito para cada señal procesada.

- RF-03: Todos los eventos clasificados como "grave" o "potencialmente grave" deben pasar obligatoriamente por revisión humana antes de que se genere ninguna notificación o alerta clínica.

- RF-04: El sistema debe generar boradores de notificación a la AEMPS en formato E2B R3 para todos los eventos notificables, requiriendo aprobación humana antes del envío.

- RF-05: El audit trail debe registrar cada acción del agente, cada herramienta invocada, cada LLM call con su prompt y respuesta completos, y cada decisión HITL, con timestamp inmutable y firma de integridad.

- RF-06: El sistema debe mantener un grafo de conocimiento actualizado en Neo4j con todas las relaciones detectadas entre medicamentos, pacientes, diagnósticos y eventos adversos.

Requisitos No Funcionales:

- RNF-01 (Explainability): Cualquier alerta generada debe poder acompañarse de una explicación en lenguaje natural comprensible por un clínico no experto en IA, accesible desde la interfaz en menos de 3 clics.

- RNF-02 (Auditabilidad): El audit trail debe conservarse durante un mínimo de 10 años, con capacidad de reconstruir el estado completo del sistema en cualquier momento pasado.

- RNF-03 (Disponibilidad): El módulo VIGIL debe mantener disponibilidad de 99,5% en horario 24/7. Los módulos ASSESS y RESPOND pueden operar con degradación graceful.

- RNF-04 (Equidad): El sistema debe ser evaluado periódicamente para detectar sesgos sistemáticos en la clasificación de señales por subpoblaciones (edad, sexo, origen).

- RNF-05 (Privacidad por Diseño): Los datos de pacientes deben ser pseudoanonimizados en todos los contextos del sistema excepto en los flujos que requieren identificación clínica específica, con acceso controlado y registrado.

2.5 Restricciones Regulatorias y de Cumplimiento

- **GDPR / LOPDGDD:** Los datos clínicos de pacientes son datos especialmente sensibles (Art. 9 GDPR). Cualquier procesamiento por parte de agentes de IA requiere base legal explícita (interés público en sanidad, Art. 9.2.h), medidas técnicas de pseudoanonimización, y registro de actividades de tratamiento (Art. 30 GDPR).

- **Regulación de IA (EU AI Act):** Un sistema de ayuda a la decisión clínica con posibilidad de influir en decisiones de prescripción se clasifica como sistema de IA de **alto riesgo** (Anexo III, categoría 5a). Esto implica: evaluación de conformidad, documentación técnica completa, registro en la base de datos de la UE, sistema de gestión de riesgos, y supervisión humana obligatoria en el ciclo de vida.

- **Normativa de Farmacovigilancia:** Real Decreto 577/2013, Regulación (UE) No 1235/2010, Directiva 2010/84/UE. El sistema que genera o facilita notificaciones a la AEMPS es parte del Sistema Español de Farmacovigilancia y debe cumplir con los plazos (15 días para RAM graves, 90 días para el resto) y formatos (E2B R3) establecidos.

- **Certificación de Producto Sanitario (MDR 2017/745):** Si el sistema se comercializara, podría requerir marcado CE como producto sanitario software (SaMD) bajo el Reglamento de Dispositivos Médicos, dependiendo de su nivel de autonomía decisional.

3. Arquitectura Empresarial

3.1 Fase Preliminar: Establecimiento del Marco

La Fase Preliminar de TOGAF 10 establece la capacidad de arquitectura antes de que comience el trabajo de arquitectura específico. Para GUARDIAN-Health, esto significa definir:

Principios de Arquitectura para Sistemas Agénticos en Salud:

- P-01 **Gobierno Integrado:** Todo componente del sistema, incluyendo cada agente y cada herramienta, opera dentro de un marco de gobierno explícito. No existen componentes sin propietario, sin política de uso y sin mecanismo de supervisión.

- P-02 **Trazabilidad Universal:** Ninguna acción del sistema, incluyendo decisiones de inferencia de LLM, puede ocurrir sin dejar rastro auditablee. La trazabilidad no es una función, es una restricción de diseño.

- P-03 **Supervisión Humana Proporcional al Riesgo:** El nivel de autonomía de cada agente es inversamente proporcional al riesgo de sus acciones. Los agentes de mayor riesgo tienen el menor nivel de autonomía.

- P-04 **Privacidad por Diseño:** La protección de datos del paciente es un requisito de diseño de primer orden, no una consideración secundaria de implementación.

- P-05 **Explicabilidad Operacional:** El sistema debe ser capaz de explicar sus decisiones en términos comprensibles para cada tipo de usuario (clínico, administrador, regulador, paciente).

- P-06 **Degradación Graceful:** El sistema debe operar de forma segura ante fallos parciales, priorizando la seguridad del paciente sobre la continuidad operacional del sistema agéntico.

Madurez de Capacidad de Arquitectura: Para la red HospitalARIA, la evaluación de madurez de arquitectura arranca en nivel 2 (Definido) para procesos clínicos pero en nivel 1 (Inicial) para sistemas de IA. El programa arquitectónico de GUARDIAN-Health incluye la elevación de la madurez de IA a nivel 3 (Gestionado) como prerrequisito de producción.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

3.2 Fase A: Visión de Arquitectura

Statement of Architecture Work:

La red HospitalARIA necesita transformar su proceso de farmacovigilancia de reactivo a proactivo, implementando un sistema de IA agéntica que monitorice continuamente señales de seguridad, evalúe su significado clínico y coordine respuestas, manteniendo supervisión humana en todas las decisiones con potencial impacto directo en la seguridad del paciente, y cumpliendo con todos los requisitos regulatorios aplicables.

Architecture Vision:

El estado objetivo en el horizonte de 18 meses es un ecosistema digital donde:

- El 100% de los pacientes ingresados están bajo monitorización activa de señales de seguridad.
- El tiempo medio de detección de señales relevantes cae de días (revisión periódica manual) a minutos (monitorización agéntica continua).
- El 100% de las notificaciones regulatorias son generadas asistidas por IA, con supervisión humana obligatoria.
- Existe un audit trail completo e inmutable de cada decisión del sistema.
- El sistema cumple con el EU AI Act y está documentado como sistema de IA de alto riesgo.

Evaluación de Capacidades

Capacidad	Estado Actual	Estado Objetivo	Gap
Monitorización de señales	Reactiva, manual	Proactiva, continua	Crítico
Evaluación de RAM	12 farmacólogos, proceso manual	IA + revisión humana	Alto
Notificación regulatoria	Proceso semiestructurado	Asistida por IA, formato E2B R3	Medio
Audit trail de decisiones	Parcial, no estructurado	Completo, inmutable	Alto
Gobierno de IA	Inexistente	Framework completo	Crítico
Interoperabilidad de datos	HCE propietarios, sin API	FHIR R4, API estándar	Alto

3.3 Fase B: Arquitectura de Negocio

Mapa de Capacidades de Negocio:

Las capacidades de negocio se organizan en cuatro dominios:

- **Dominio Clínico:** Prescripción y dispensación de medicamentos, monitorización clínica, gestión de incidentes de seguridad, coordinación asistencial.
- **Dominio de Farmacovigilancia:** Detección de señales, evaluación de causalidad, gestión de notificaciones regulatorias, comunicación con autoridades, educación a prescriptores.
- **Dominio de Gobierno de IA:** Definición de políticas de agentes, supervisión de decisiones, gestión de modelos, monitorización de sesgos, auditoría del sistema.
- **Dominio de Datos y Compliance:** Gestión de datos clínicos, cumplimiento GDPR, gestión de accesos, reporting regulatorio, conservación de evidencias.

Flujos de Valor Afectados:

El flujo de valor principal que GUARDIAN-Health transforma es la cadena **Señal** → **Detección** → **Evaluación** → **Respuesta** → **Aprendizaje**. En el modelo actual, cada paso es manual y desconectado. En el modelo objetivo, los agentes automatizan la detección y la evaluación preliminar, mientras los humanos se concentran en las decisiones de mayor complejidad clínica y las responsabilidades regulatorias.

Roles y Responsabilidades (RACI para decisiones clave):

Decisión	Agente VIGIL	Agente ASSESS	Farmacólogo	Director Médico	DCF
Detección de señal bruta	R	-	I	-	I
Clasificación de gravedad	-	R	A	-	I
Activación de HITL	-	R	-	A	I
Aprobación de notificación AEMPS	-	-	R	A	C
Cierre del expediente	-	-	R	I	A

3.4 Fase C: Arquitectura de Sistemas de Información

Arquitectura de Aplicaciones:

La arquitectura de aplicaciones de GUARDIAN-Health sigue un patrón de capas con separación clara de responsabilidades:

- **Capa de Adquisición de Datos:** Conectores FHIR R4, adaptadores HL7 v2, listeners de Solace Agent Mesh. Transforma datos de sistemas fuente a la representación interna del sistema.

- **Capa de Procesamiento Agéntico:** LangGraph (Orquestación de flujo con estado), CrewAI (Crews de evaluación), AutoGen (evaluación adversarial). Implementa la lógica de los tres módulos VIGIL, ASSESS y RESPOND.

- **Capa de Datos:** Neo4j (grafo de conocimiento clínico), PostgreSQL (datos transaccionales y audit trail), Pinecone/Qdrant (índice vectorial de literatura médica).

- **Capa de Integración:** FastAPI (APIs REST), Solace Agent Mesh (mensajería asíncrona), n8n (workflows de automatización).

- **Capa de Presentación y HITL:** Dashboard clínico (FastAPI + frontend), consola de gobierno (métricas y alertas del sistema), portal de farmacovigilancia.

- **Capa de Observabilidad:** LangSmith (trazas agénticas), logging estructurado, métricas de sistema.

Arquitectura de Datos de Aplicación:

Los datos fluyen en un patrón event-driven: los sistemas clínicos publican eventos en Solace, el Módulo VIGIL los consume y procesa, las señales detectadas se publican como nuevos eventos, el Módulo ASSESS los consume y produce evaluaciones, y el Módulo RESPOND actúa sobre las evaluaciones. En cada paso, los datos se persisten con su provenance completo.

3.5 Fase D: Arquitectura Tecnológica

Stack Tecnológico Completo:

```
┌──────────────────────────────────────────────────────────────┐
│                      CAPA DE PRESENTACIÓN                    │
│   FastAPI Dashboard Clínico │ Portal Farmacovigilancia │ n8n  │
├──────────────────────────────────────────────────────────────┤
│                  CAPA DE ORQUESTACIÓN AGÉNTICA               │
│   LangGraph (Flujo Agéntico) │ CrewAI (Crews) │ AutoGen (Eval) │
│              LangChain (Abstracción LLM + Tools)             │
├──────────────────────────────────────────────────────────────┤
│                     CAPA DE INFERENCIA                       │
│      Groq API (llama-3.3-70b-versatile / llama-3.1-8b-instant) │
│      HuggingFace (nomic-embed-text-v1, embeddings locales)   │
├──────────────────────────────────────────────────────────────┤
│                     CAPA DE INTEGRACIÓN                      │
│   Solace Agent Mesh (Eventos) │ FastAPI (REST) │ n8n (Workflows)│
├──────────────────────────────────────────────────────────────┤
│                       CAPA DE DATOS                          │
│  Neo4j (Grafo Clínico) │ PostgreSQL (Transaccional + Audit)  │
│  Pinecone (Vector Prod) │ Qdrant (Vector Dev)                │
├──────────────────────────────────────────────────────────────┤
│                     CAPA DE OBSERVABILIDAD                   │
│   LangSmith (Trazas Agénticas) │ Logging Estructurado │ Métricas│
└──────────────────────────────────────────────────────────────┘
```

Decisiones de Infraestructura:

Todo el stack se ejecuta en contenedores Docker (con `docker-compose` para el entorno de desarrollo) que puede desplegarse en Kubernetes para producción. Esta decisión garantiza reproducibilidad del entorno, aislamiento de servicios, y la capacidad de escalar componentes independientemente.

Los LLMs no se ejecutan localmente (excepto para pruebas de unidad con modelos pequeños via Ollama opcional). La inferencia se delega a Groq, lo que significa que los datos enviados al LLM deben estar pseudoanonimizados antes de salir del perímetro controlado.

3.6 Gobierno de Arquitectura y ADRs

Las Decisiones de Arquitectura son registros formales de por qué se tomó cada decisión técnica significativa. Son herramientas de gobierno: permiten que futuros arquitectos entiendan el razonamiento detrás de cada elección y evalúen su vigencia a medida que el contexto cambia. Ver Apéndice A para los ADRs completos del proyecto.

4. Arquitectura de Datos

4.1 Gobierno del Dato: Estructura y Responsabilidades

DAMA DMBOK2 define el Gobierno del Dato como "el ejercicio de autoridad, control y toma de decisiones compartida (planificación, seguimiento y aplicación) sobre la gestión de datos". Para GUARDIAN-Health, este marco se traduce en una estructura de gobierno específica para datos clínicos en un contexto de IA agéntica.

Consejo de Gobierno del Dato (CGD):

El CGD es el órgano de gobierno supremo para los datos de GUARDIAN-Health. Se compone de:

- Data Owner (Propietario del Dato): Director Médico. Responsable último de los datos clínicos del sistema.
- Data Steward Clínico: Jefe del Departamento de Farmacovigilancia. Responsable de la calidad y uso correcto de los datos en el contexto clínico.
- Data Steward Técnico: Arquitecto de Datos. Responsable de la implementación técnica de las políticas de datos.
- Data Privacy Officer (DPO): Delegado de Protección de Datos. Responsable del cumplimiento GDPR.
- Data Quality Lead: Responsable de los procesos de calidad del dato.

Dominios de Datos del Sistema:

GUARDIAN-Health gestiona cinco dominios de datos diferenciados, cada uno con su política de gobierno específica:

- **Dominio Clínico-Paciente:** Datos personales de salud. Clasificación: Altamente Confidencial. Acceso: Solo personal sanitario autorizado con relación asistencial. Retención: 15 años (normativa sanitaria española). Pseudoanonimización obligatoria para uso en agentes IA.

- **Dominio Farmacológico:** Datos de medicamentos, prescripciones, dispensaciones. Clasificación: Confidencial. Acceso: Clínicos, farmacólogos, sistema agéntico (sin restricción de pseudoanonimización). Retención: 10 años.
    -
- **Dominio de Eventos Adversos:** Señales detectadas, evaluaciones, notificaciones. Clasificación: Altamente Confidencial. Acceso: DCF, clínico responsable, reguladores. Retención: 20 años (requisito regulatorio de farmacovigilancia).

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

- **Dominio de Decisiones Agénticas:** Audit trail de acciones del sistema, trazas de LLM, estados de flujo. Clasificación: Confidencial-Interno. Acceso: Auditores, administradores del sistema, DPO. Retención: 10 años.

- **Dominio de Conocimiento Médico:** Literatura indexada, guías clínicas, fichas técnicas. Clasificación: Público/Licenciado. Acceso: Sin restricciones dentro del sistema. Retención: Actualización continua.

Políticas de Gobierno:

- PD-01: Ningún dato de paciente identificable puede ser enviado a un servicio de LLM externo. Todos los contextos enviados a Groq deben usar pseudónimos de paciente (UUID) y deben haber sido revisados por la función de sanitización de datos.

- PD-02: Cada campo de dato en el sistema debe tener un Data Steward asignado, documentado en el catálogo de datos.

- PD-03: Cualquier cambio en el esquema de datos del Dominio Clínico-Paciente o del Dominio de Eventos Adversos requiere aprobación del CGD antes de implementación.

- PD-04: Los datos del Dominio de Decisiones Agénticas son inmutables una vez registrados. No pueden ser modificados ni eliminados, solo marcados como corregidos con el registro de la corrección añadido.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

4.2 Arquitectura de Datos y Modelo Conceptual

Entidades Principales del Dominio:

```
PACIENTE ─── tiene ─── INGRESO
    │                      │
    │                  tiene
    │                      │
    │                  PRESCRIPCIÓN ─── incluye ─── MEDICAMENTO
    │                      │                              │
    │                  genera                         tiene
    │                      │                              │
    └── experimenta ─── EVENTO_ADVERSO            FICHA_TÉCNICA
                           │
                       es_evaluado_por
                           │
                       EVALUACIÓN_AGÉNTICA
                           │
                     ┌─────┴─────┐
                     │           │
              SEÑAL_BRUTA     NOTIFICACIÓN
              (Módulo VIGIL)  (AEMPS / E2B R3)
```

Modelo de Grafo en Neo4j:

-- Nodos principales

- `(:Paciente {id_pseudo, edad, sexo, grupo_etnico})`
- `(:Medicamento {principio_activo, nombre_comercial, atc_code, via_administracion})`
- `(:Prescripcion {id, fecha_inicio, fecha_fin, dosis, frecuencia})`
- `(:Diagnostico {codigo_cie10, descripcion})`
- `(:EventoAdverso {id, fecha_deteccion, tipo, gravedad, estado})`
- `(:EvaluacionAgéntica {id, fecha, nivel_confianza, consenso_agentes})`
- `(:Notificacion {id, fecha, tipo, estado, numero_caso_aemps})`

-- Relaciones

- `(:Paciente)-[:RECIBE]->(:Prescripcion)`
- `(:Prescripcion)-[:INCLUYE]->(:Medicamento)`
- `(:Paciente)-[:TIENE]->(:Diagnostico)`
- `(:Paciente)-[:EXPERIMENTA]->(:EventoAdverso)`
- `(:EventoAdverso)-[:POSIBLEMENTE_CAUSADO_POR]->(:Medicamento)`
- `(:EventoAdverso)-[:ES_EVALUADO_POR]->(:EvaluacionAgéntica)`
- `(:EvaluacionAgéntica)-[:GENERA]->(:Notificacion)`

- `(:Medicamento)-[:INTERACCIONA_CON]->(:Medicamento)`
- `(:Medicamento)-[:CONTRAINDICADO_EN]->(:Diagnostico)`

Modelo Relacional en PostgreSQL:

Las tablas principales del esquema relacional cubren:

- `pacientes` (datos maestros pseudoanonimizados)
- `ingresos`, `prescripciones`, `resultados_laboratorio`
- `senales_brutas`, `evaluaciones`, `notificaciones`
- `audit_log` (inmutable, append-only)
- `agent_decisions` (registro de cada decisión agéntica con traza)
- `hitl_reviews` (registro de revisiones humanas con decisión y justificación)
- `langgraph_checkpoints` (estado persistente del grafo de agentes).

4.3 Gestión de Datos Maestros y de Referencia

Los datos maestros del sistema incluyen el catálogo de medicamentos (con códigos ATC y Nacional), la tabla de diagnósticos (CIE-10), el catálogo de personal sanitario (para asignación de responsabilidades), y los centros hospitalarios de la red.

Estos datos se gestionan con un proceso de Master Data Management (MDM) simplificado:

- Fuente de verdad única por entidad maestra.
- Proceso de sincronización documentado con los sistemas fuente.
- Versioning de registros maestros para permitir reconstrucción histórica.

Los datos de referencia críticos para el funcionamiento de los agentes son las guías clínicas de farmacovigilancia, las bases de datos de interacciones farmacológicas (simuladas con datos sintéticos), y los criterios de clasificación de gravedad de la EMA. Estos datos se cargan en Pinecone/Qdrant para acceso semántico por los agentes.

4.4 Calidad del Dato en Contextos Agénticos

La calidad del dato en un sistema agéntico tiene una dimensión adicional respecto a sistemas tradicionales: la calidad del dato de entrada afecta directamente a la calidad del razonamiento del agente. Un agente que razona sobre datos incorrectos puede producir evaluaciones plausibles pero erróneas, y la naturaleza del sistema hace que estos errores sean difíciles de detectar sin monitorización activa.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

Dimensiones de Calidad Gestionadas:

- DQ-01 **Completitud:** Los datos enviados a los agentes deben tener un nivel mínimo de completitud. El sistema verifica, antes de activar el Módulo ASSESS, que los datos del paciente, la prescripción y el evento tienen los campos requeridos para la evaluación.

- DQ-02 **Puntualidad:** En farmacovigilancia, un dato correcto pero tardío puede ser inútil. El sistema monitoriza la latencia de cada fuente de datos y degrada el nivel de confianza de las evaluaciones basadas en datos con latencia superior a umbrales definidos.

- DQ-03 **Consistencia:** Los datos del mismo paciente provenientes de distintos sistemas (HCE, farmacia, laboratorio) deben ser reconciliados. El sistema mantiene un proceso de reconciliación de identidades de paciente usando el grafo Neo4j.

- DQ-04 **Precisión contextual:** Esta dimensión es específica de los sistemas agénticos. Mide si los datos en el contexto del agente son representativos de la situación clínica real. Se monitoriza mediante la tasa de acuerdo entre la evaluación agéntica y la revisión humana posterior.

Framework de Calidad para Datos de Agentes:

Antes de incluir cualquier dato en el contexto de un agente (el "prompt"), el sistema ejecuta un proceso de validación que verifica completitud de campos requeridos, consistencia con datos históricos del mismo paciente, ausencia de datos identificativos directos (pseudoanonimización), y frescura del dato según la política de latencia máxima.

4.5 Linaje, Metadatos y Trazabilidad

El data lineage en GUARDIAN-Health es bidireccional: para cada señal detectada por el sistema, se puede trazar hacia atrás hasta los datos fuente que la originaron. Y para cada dato fuente, se puede trazar hacia adelante hasta las decisiones agénticas en las que influyó.

Metadatos Mínimos Requeridos:

Cada dato en el sistema lleva consigo un conjunto mínimo de metadatos:

- `source_system`: Sistema que generó el dato
- `ingestion_timestamp`: Momento de ingesta en el sistema
- `data_as_of`: Momento al que se refiere el dato (puede diferir del ingestion_timestamp)
- `quality_score`: Puntuación de calidad calculada en el momento de ingesta
- `steward`: Data Steward responsable del dato
- `classification`: Clasificación de confidencialidad
- `used_in_agent_contexts`: Lista de contextos agénticos donde este dato fue incluido (para auditabilidad).

Catálogo de Datos:

El catálogo de datos se implementa en PostgreSQL y documenta cada entidad de datos del sistema con su definición de negocio, propietario, clasificación, esquema técnico, y lineage. El catálogo es la fuente de verdad para los auditores y el DPO.

4.6 Pipeline de Datos: Arquitectura Medallion en Salud

El pipeline de datos sigue la arquitectura Medallion (Bronze, Silver, Gold) adaptada al contexto clínico:

- **Bronze Layer (Datos Brutos):** Datos tal como llegan de los sistemas fuente, sin transformación. Incluyen mensajes HL7 v2 completos, respuestas FHIR crudas, exports de sistemas de laboratorio en formato propietario. Esta capa se conserva para auditoría forense y reproducibilidad. Los datos de esta capa nunca se usan directamente en agentes.

- **Silver Layer (Datos Limpios y Conformados):** Datos normalizados al modelo conceptual del sistema, validados en calidad, pseudoanonimizados, y enriquecidos con referencias cruzadas (códigos ATC, CIE-10 canónicos). Esta capa alimenta Neo4j y las tablas relacionales de PostgreSQL. Los agentes de monitorización (Módulo VIGIL) consumen datos de esta capa.

- **Gold Layer (Datos Agénticos):** Datos preparados específicamente para el consumo de agentes: contextos pre-construidos, resúmenes de historial, señales estructuradas. También

incluye los embeddings vectoriales de documentación médica indexada en Pinecone/Qdrant. Los agentes de evaluación (Módulo ASSESS) consumen datos de esta capa.

4.7 Privacidad, Seguridad y Cumplimiento del Dato

Pseudoanonimización:

El sistema utiliza un esquema de pseudoanonimización de dos capas. La capa externa asigna un UUID permanente a cada paciente (almacenado en una tabla de mapping con acceso muy restringido). La capa interna genera identificadores de sesión temporales para cada contexto de agente, que expiran y son rotados. Los datos enviados a Groq utilizan solo el identificador de sesión, que no puede ser re-vinculado al paciente sin acceder al sistema de pseudoanonimización.

Control de Acceso:

El sistema implementa control de acceso basado en roles (RBAC) con granularidad a nivel de campo para datos de paciente. Los agentes tienen roles de acceso específicos que les permiten solo los datos necesarios para su función (principio de mínimo privilegio).

Retención y Borrado:

Las políticas de retención se implementan como procesos automatizados con supervisión humana. El derecho al olvido del GDPR se implementa mediante pseudoanonimización reforzada (destrucción de la clave de mapping) en lugar de borrado físico, para mantener la utilidad estadística de los datos históricos.

5. Arquitectura de IA Agéntica

5.1 Patrones de Agente y Justificación de Selección

Para GUARDIAN-Health, se emplean cuatro patrones agénticos diferenciados, seleccionados por su adecuación a los requisitos de gobierno y las características de cada módulo.

Patrón ReAct (Reasoning + Acting): Es el patrón base para los agentes del Módulo VIGIL. El agente alterna entre razonamiento (interpretar los datos disponibles) y acción (invocar herramientas para obtener más datos). Este patrón es adecuado para la fase de detección porque es transparente: el razonamiento del agente está explícito en cada paso, lo que facilita la auditabilidad.

Patrón Plan-and-Execute: Empleado en el Módulo ASSESS para la evaluación de señales complejas. El agente primero produce un plan explícito de evaluación (qué información necesita, qué criterios aplicará, qué expertos simulados consultará), luego ejecuta cada paso del plan. Este patrón mejora la coherencia de evaluaciones complejas y produce un artefacto de razonamiento (el plan) que puede ser revisado por un humano.

Patrón Multi-Agent Debate: Implementado via AutoGen en el Módulo ASSESS para casos de clasificación ambigua. Dos agentes adoptan posiciones opuestas (agente proponente de clasificación grave, agente escéptico que cuestiona la causalidad) y debaten estructuradamente. El proceso de debate y la síntesis final quedan registrados como evidencia del razonamiento.

Patrón Supervisor-Worker: Empleado en el Módulo RESPOND con LangGraph. Un agente Supervisor coordina workers especializados (redactor de notificaciones, agente de comunicación a clínico, agente de actualización de grafo). El Supervisor mantiene el estado global y toma decisiones de routing.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

5.2 Topología Multi-Agente: Roles y Responsabilidades

```
┌──────────────────────────────────────────────────────────────────┐
│                    MÓDULO VIGIL (LangGraph)                      │
│                                                                  │
│   ┌──────────────────┐     ┌──────────────────┐                  │
│   │ Agente Monitor   │     │ Agente Monitor   │                  │
│   │ Medicamentos     │     │ Laboratorio      │                  │
│   └──────────────────┘     └──────────────────┘                  │
│            └──────────────────┬──────────────────┘               │
│                               ▼                                  │
│                ┌──────────────┐                                  │
│                │ Agente       │                                  │
│                │ Correlación  │ ── publica señales brutas ──►    │
│                └──────────────┘                                  │
└──────────────────────────────────────────────────────────────────┘
                              │ Solace Agent Mesh
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                   MÓDULO ASSESS (CrewAI + AutoGen)               │
│                                                                  │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐    │
│   │ Agente       │   │ Agente       │   │ Agente           │    │
│   │ Farmacólogo  │   │ Clínico      │   │ Regulatorio      │    │
│   └──────────────┘   └──────────────┘   └──────────────────┘    │
│          └──────────────┬──────────────────┬──────────────┘      │
│                         ▼                                        │
│                     ┌──────────┐                                 │
│                     │ Agente   │                                 │
│                     │ Síntesis │ ──► Evaluación estructurada     │
│                     └──────────┘                                 │
│                          │                                       │
│                     ¿Alta gravedad?──► HITL Obligatorio          │
└──────────────────────────────────────────────────────────────────┘
                              │ (post-HITL si aplica)
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                  MÓDULO RESPOND (LangGraph)                      │
│                                                                  │
│   ┌──────────────┐                                               │
│   │ Agente       │                                               │
│   │ Supervisor   ├──► Agente Notificador Clínico                 │
│   │              ├──► Agente Redactor AEMPS (E2B R3)             │
│   │              └──► Agente Actualizador Neo4j                  │
│   └──────────────┘                                               │
└──────────────────────────────────────────────────────────────────┘
```

5.3 Orquestación con LangGraph

LangGraph implementa el Módulo VIGIL y el Módulo RESPOND como grafos de estado. El diseño del grafo de estado es la decisión de gobierno más importante de la arquitectura agéntica.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

Estado del Grafo (TypedDict):

```
class GuardianState(TypedDict):
    # Identidad y trazabilidad
    session_id: str                     # UUID de la sesión agéntica
    patient_pseudo_id: str              # ID pseudoanonimizado del paciente
    trace_id: str                       # ID de traza LangSmith

    # Datos clínicos de contexto
    clinical_context: dict              # Datos sanitizados del paciente
    prescriptions_active: list          # Prescripciones activas pseudoanonimizadas
    lab_results_recent: list            # Resultados recientes de laboratorio

    # Señales y evaluaciones
    raw_signals: list[RawSignal]        # Señales brutas detectadas
    evaluation: Optional[Evaluation]    # Evaluación del Módulo ASSESS

    # Control de flujo y gobierno
    requires_hitl: bool                 # ¿Requiere revisión humana?
    hitl_decision: Optional[str]        # Decisión humana si aplica
    hitl_reviewer_id: Optional[str]     # ID del revisor humano
    hitl_timestamp: Optional[str]       # Timestamp de la revisión

    # Audit trail
    agent_actions: list[AgentAction]    # Registro de cada acción de agente
    tool_calls: list[ToolCall]          # Registro de cada herramienta invocada
    llm_calls: list[LLMCall]            # Registro de cada llamada a LLM

    # Estado del flujo
    current_module: str                 # Módulo activo
    flow_status: str                    # Estado del flujo
    error_state: Optional[str]          # Error si existe
```

El estado incluye explícitamente el audit trail como parte del estado del grafo. Esto no es un logging secundario: es el mecanismo de gobierno integrado en el flujo agéntico.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

Nodos del Grafo VIGIL:

```
# Nodos del grafo
       vigil_graph.add_node("initialize", initialize_node)
       vigil_graph.add_node("fetch_clinical_context", fetch_context_node)
       vigil_graph.add_node("sanitize_data", sanitize_data_node)
       vigil_graph.add_node("monitor_prescriptions", monitor_prescriptions_node)
       vigil_graph.add_node("monitor_lab_results", monitor_lab_node)
       vigil_graph.add_node("correlate_signals", correlate_signals_node)
       vigil_graph.add_node("publish_signals", publish_signals_node)
       vigil_graph.add_node("update_audit_trail", update_audit_node)
       # Edges condicionales para gobierno
       vigil_graph.add_conditional_edges(
           "correlate_signals",
           route_by_signal_quality,  # Verifica calidad mínima antes de publicar
           {
               "publish": "publish_signals",
               "low_quality_halt": "update_audit_trail"  # Documenta el halt
           }
       )
```

Checkpointing para Gobierno:

El checkpointer de LangGraph persiste el estado completo en PostgreSQL después de cada nodo. Esto permite:

- Retomar un flujo interrumpido desde el último checkpoint.
- Auditar el estado exacto del sistema en cualquier momento del proceso.
- Reproducir el flujo completo para investigación forense.
- Implementar HITL como una interrupción del grafo (el grafo se pausa, espera la decisión humana, y retoma).

5.4 Crews con CrewAI y AutoGen

Crew de Evaluación (CrewAI):

```
# Definición del crew de evaluación ASSESS
assessment_crew = Crew(
    agents=[
        pharmacologist_agent,   # Evalúa plausibilidad biológica
        clinician_agent,        # Evalúa contexto del paciente
        regulatory_agent,       # Evalúa notificabilidad
        synthesis_agent         # Produce evaluación final
    ],
    tasks=[
        causality_assessment_task,
        severity_classification_task,
        notifiability_determination_task,
        final_evaluation_task
    ],
    process=Process.sequential,
    verbose=True,
    output_log_file="crew_assessment_log.json"  # Gobierno: log de crew
)
```

Cada tarea del crew produce una salida tipada con Pydantic. El modelo de salida de `final_evaluation_task` incluye el nivel de confianza, la síntesis del razonamiento de cada agente, las discrepancias entre agentes (si las hay), y la recomendación final con su justificación.

Debate Adversarial (AutoGen):

Para señales con evaluaciones discordantes entre agentes del crew, se activa un proceso de debate AutoGen. Un agente adopta el rol de "defensor de la clasificación propuesta" y otro el de "evaluador crítico". El debate sigue un protocolo estructurado con un número máximo de rondas y criterios de terminación definidos. El transcript completo del debate se almacena como evidencia de razonamiento.

5.5 Diseño de Memoria: Corto, Largo Plazo y Episódica

- **Memoria de Corto Plazo:** El estado del grafo LangGraph es la memoria de corto plazo del sistema. Contiene el contexto activo de la sesión agéntica actual.

- **Memoria de Largo Plazo:** Neo4j es la memoria de largo plazo relacional del sistema. Contiene el grafo acumulado de conocimiento clínico: relaciones entre medicamentos, pacientes, diagnósticos y eventos adversos. Los agentes pueden consultar el grafo para encontrar patrones similares en el historial de la red hospitalaria.

- **Memoria Episódica:** Pinecone/Qdrant almacena embeddings de evaluaciones anteriores, guías clínicas, y literatura médica relevante. Los agentes usan búsqueda semántica para recuperar episodios similares (evaluaciones anteriores de señales parecidas) que enriquecen su razonamiento.

- **Memoria de Trabajo del Agente:** Para cada invocación de LLM, el contexto se construye explícitamente combinando elementos de las tres memorias anteriores, siguiendo una plantilla de construcción de contexto que garantiza que (a) el contexto no supera los límites del modelo, (b) los datos están correctamente pseudoanonimizados, y (c) los metadatos de provenance están incluidos.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

5.6 Human-in-the-Loop (HITL): Diseño y Justificación

El HITL no es una concesión a la desconfianza en los agentes: es un componente de arquitectura de primer orden, definido por los requisitos regulatorios y los principios de gobierno.

Matriz de Decisión HITL:

Tipo de Acción	Autonomía del Agente	Nivel de HITL	Justificación
Detección de señal bruta	Completa	Ninguno	Bajo riesgo, alta frecuencia, revisable
Clasificación de señal como leve	Completa	Notificación informativa	Impacto reversible
Clasificación como moderada	Propuesta	Confirmación 24h	Requiere validación clínica
Clasificación como grave	Borrador	Aprobación previa	Alto impacto, no reversible
Notificación a AEMPS	Borrador	Aprobación obligatoria + firma	Acto regulatorio
Comunicación directa al paciente	Prohibida	N/A	Fuera del alcance del sistema

Implementación del HITL en LangGraph:

El nodo de evaluación de HITL interrumpe el grafo si es necesario

```
def check_hitl_requirement(state: GuardianState) -> str:
    evaluation = state["evaluation"]
    if evaluation.severity in ["GRAVE", "POTENCIALMENTE_GRAVE"]:
        state["requires_hitl"] = True
        return "hitl_required"  # El grafo se interrumpe aquí
    return "proceed_to_respond"
# La interrupción se implementa con interrupt_before
guardian_graph = StateGraph(GuardianState)
guardian_graph.add_node("await_hitl_decision", await_hitl_node)
# LangGraph pausa el grafo en este nodo hasta que se llame a `resume`
# con la decisión humana como input
```

La decisión humana se registra en el estado del grafo con: ID del revisor, timestamp, decisión (aprobar / rechazar / modificar), y justificación libre. Este registro es parte del audit trail permanente.

5.7 Herramientas y APIs Externas

Los agentes del sistema tienen acceso a un conjunto de herramientas específicamente diseñadas y aprobadas. No tienen acceso a herramientas genéricas de búsqueda en internet o ejecución de código arbitrario.

Herramientas de Datos Clínicos:

- `get_patient_context(pseudo_id)`: Recupera el contexto clínico pseudoanonimizado del paciente de Silver Layer.
- `get_active_prescriptions(pseudo_id)`: Retorna prescripciones activas con codificación ATC.
- `get_lab_results(pseudo_id, days=7)`: Resultados de laboratorio recientes con valores de referencia.
- `query_clinical_graph(cypher_query, params)`: Ejecuta consultas Cypher en Neo4j (con whitelist de operaciones permitidas).

Herramientas de Conocimiento Médico:

- `search_pharmacovigilance_guidelines(query)`: Búsqueda semántica en guías de la EMA/AEMPS.
- `search_similar_cases(signal_description)`: Recupera evaluaciones previas similares de Pinecone/Qdrant.
- `get_drug_interactions(atc_codes)`: Consulta la base de datos de interacciones (local, no externa).

Herramientas de Gestión de Flujo:

- `classify_severity(event_description, criteria)`: Aplica criterios de clasificación estándar.
- `generate_e2b_draft(evaluation)`: Genera borrador de notificación E2B R3.
- `notify_clinician(clinician_id, urgency, message)`: Envía notificación (con registro de auditoría).
- `update_knowledge_graph(entities, relations)`: Actualiza Neo4j con las entidades detectadas.

Todas las herramientas tienen logging automático: cada invocación registra los parámetros de entrada, la salida, el tiempo de ejecución, y el agente que la invocó.

5.8 Mensajería Orientada a Eventos: Solace Agent Mesh

Solace Agent Mesh actúa como el tejido de comunicación entre los tres módulos. Los tópicos de mensajería siguen una jerarquía estructurada:

```
guardian/v1/signals/raw/{hospital_id}/{patient_pseudo_id}

guardian/v1/signals/evaluated/{severity}/{patient_pseudo_id}

guardian/v1/hitl/required/{case_id}

guardian/v1/hitl/decision/{case_id}

guardian/v1/notifications/clinician/{clinician_id}

guardian/v1/notifications/aemps/{case_id}

guardian/v1/audit/{module}/{event_type}
```

La jerarquía de tópicos tiene significado de gobierno: los tópicos `guardian/v1/audit/` son de solo-escritura para los módulos y de solo-lectura para el sistema de auditoría. Los tópicos `guardian/v1/hitl/` requieren autenticación específica de rol humano para el tópico `decision`.

5.9 Observabilidad Agéntica: LangSmith + Trazas de Agente

LangSmith captura automáticamente las trazas de todas las cadenas LangChain y grafos LangGraph. Para GUARDIAN-Health, la configuración de LangSmith es parte del framework de gobierno, no solo del debugging.

Metadatos de Traza:

Cada traza se enriquece con metadatos de gobierno:

```
with trace(
    project_name="guardian-health",
    tags=["module:vigil", "hospital:madrid"],
    metadata={
        "patient_pseudo_id": state["patient_pseudo_id"],
        "session_id": state["session_id"],
        "data_classification": "confidential",
        "hitl_required": state["requires_hitl"],
        "regulatory_case_id": state.get("regulatory_case_id")
    }
):

# Ejecución del agente
```

Estas trazas son consultables y constituyen parte del audit trail regulatorio del sistema.

6. Gobierno y Responsabilidad por Diseño

6.1 Por qué el Gobierno ha de ser desde el Día Cero

Esta es la tesis central de GUARDIAN-Health, y vale la pena articularla con precisión técnica, porque la industria tiende a tratarla como un problema de gestión en lugar de como lo que realmente es: un problema de arquitectura.

El gobierno retroactivo no escala.

Cuando se añade gobierno a un sistema ya construido, se añade como una capa de instrumentación externa: logs de aplicación, dashboards de monitorización, revisiones periódicas de salidas. Esta instrumentación externa tiene un problema fundamental: no puede capturar el razonamiento interno del sistema. Puede decirte qué decidió el agente, pero no por qué razonó de esa manera, con qué datos, qué alternativas consideró, y qué factores determinaron su elección. En sistemas de IA agéntica, el "por qué" es exactamente lo que los auditores, los reguladores, y los médicos necesitan entender.

El gobierno integrado desde el diseño captura el razonamiento en el momento en que ocurre, como parte del estado del sistema, no como un artefacto secundario. Esto es lo que hace el diseño del estado del grafo de LangGraph en GUARDIAN-Health: el audit trail no es un sistema aparte que observa el agente, es parte del estado del agente.

El desfase de confianza se acumula exponencialmente.

Un sistema agéntico en un dominio de alto riesgo sin gobierno visible genera un desfase de confianza entre los usuarios (clínicos, reguladores) y el sistema que crece con cada decisión no explicada. Recuperar esa confianza requiere demostrar no solo que el sistema funciona correctamente ahora, sino que ha funcionado correctamente desde el principio. Sin audit trail desde el inicio, esa demostración es imposible.

La deuda de gobierno tiene un punto de no retorno.

En sistemas de software tradicionales, la deuda técnica se puede saldar con refactoring. En sistemas de IA agéntica con historial decisional, la deuda de gobierno no se puede saldar retrospectivamente: no se pueden recrear las trazas de decisiones que no se registraron, no se puede reconstruir el linaje de datos que no se capturó. El coste del gobierno desde el día cero es la inversión en diseño. El coste del gobierno retroactivo, cuando es posible, es habitualmente mayor que el coste de haber diseñado el sistema correctamente desde el inicio. Y en muchos casos, simplemente no es posible.

El EU AI Act hace del gobierno una restricción legal, no una práctica recomendada.

Para sistemas de IA de alto riesgo (categoría en la que caería GUARDIAN-Health), el EU AI Act requiere documentación técnica completa, sistema de gestión de riesgos, registro de trazabilidad (Art. 12), supervisión humana (Art. 14), y evaluación de conformidad antes del despliegue. Estas obligaciones son verificables solo si el sistema fue diseñado para producir la evidencia requerida. No se puede construir el sistema primero y luego intentar demostrar conformidad si el sistema no fue diseñado para ello.

6.2 Riesgos de Ausencia de Gobierno Agéntico Temprano

Riesgo 1: Opacidad Decisional Acumulativa

Sin estructuras de trazabilidad diseñadas desde el inicio, las decisiones de los agentes se acumulan en "cajas negras": el sistema produce outputs, pero la cadena de razonamiento que llevó a cada output no está disponible para revisión. En farmacovigilancia, donde una clasificación incorrecta puede tener consecuencias clínicas, esta opacidad es inaceptable desde el primer día de operación.

Riesgo 2: Deriva de Comportamiento No Detectada

Los sistemas agénticos pueden experimentar deriva en su comportamiento a lo largo del tiempo (model drift, prompt drift, context drift) de forma gradual e imperceptible. Sin monitorización de gobierno integrada, esta deriva no se detecta hasta que sus consecuencias son observables: una tasa creciente de evaluaciones incorrectas, un sesgo sistemático en las clasificaciones, un aumento en el tiempo de procesamiento. Con gobierno integrado, los indicadores de deriva son visibles en tiempo real.

Riesgo 3: Responsabilidad Jurídica Indeterminable

Cuando un sistema de IA agéntica contribuye a un evento adverso no detectado o a una notificación regulatoria incorrecta, la pregunta legal central es: ¿quién es responsable y en qué medida contribuyó el sistema a ese resultado? Sin audit trail de las decisiones del agente, de los datos que procesó, y de las supervisiones humanas realizadas, esta pregunta no tiene respuesta verificable. La responsabilidad queda indeterminada, lo que expone a todos los actores (el hospital, el fabricante del software, los clínicos que actuaron sobre las alertas) a litigios sin resolución clara.

Riesgo 4: Imposibilidad de Cumplimiento Regulatorio

El EU AI Act, el GDPR y la normativa de farmacovigilancia requieren formas de documentación que no se pueden generar retroactivamente. Si el sistema no registra, desde el inicio, los datos procesados, los modelos utilizados, las decisiones tomadas y las supervisiones realizadas, no puede demostrar cumplimiento cuando el regulador lo requiera, que puede ser en cualquier momento.

Riesgo 5: Erosión del Capital de Confianza Clínica

Los médicos son usuarios exigentes y escépticos. Si un sistema de IA produce alertas que no pueden ser explicadas de forma comprensible, los médicos aprenden a ignorarlas. La tasa de ignorancia de alertas (alert fatigue) en sistemas de decisión clínica sin explicabilidad adecuada puede superar el 90% en cuestión de semanas. Un sistema de farmacovigilancia que sus usuarios ignoran es peor que no tener sistema.

Riesgo 6: Amplificación de Sesgos sin Control

Los LLMs heredan sesgos de sus datos de entrenamiento. Un agente que evalúa la gravedad de eventos adversos puede tener sesgos sistemáticos relacionados con características del paciente. Sin monitorización de equidad integrada desde el inicio, estos sesgos se amplifican con cada ciclo operacional: el sistema actúa sobre sus evaluaciones sesgadas, esas acciones generan datos, y esos datos refuerzan el sesgo en los ciclos siguientes.

6.3 Framework de Gobierno para Sistemas Agénticos

GUARDIAN-Health implementa un framework de gobierno en cuatro capas que cubre desde el diseño hasta la operación continua.

Capa 1: Gobierno de Diseño

Las políticas de gobierno se definen como restricciones de arquitectura antes de que se escriba una línea de código. Incluyen: la matriz HITL (qué decisiones requieren supervisión humana y de qué tipo), las políticas de datos (qué datos pueden acceder los agentes y en qué condiciones), las restricciones de herramientas (qué herramientas tienen acceso los agentes), y los principios de diseño de estado (qué información debe estar en el estado del grafo para garantizar auditabilidad).

Capa 2: Gobierno de Datos

Implementa el framework DAMA DMBOK2 detallado en el capítulo 4. Cubre la clasificación de datos, las políticas de acceso, la calidad del dato, el lineage, y los procesos de retención y destrucción segura.

Capa 3: Gobierno Operacional

Monitorización continua de las métricas de gobierno del sistema: tasa de acuerdo agente-humano en evaluaciones HITL, tasa de evaluaciones con confianza baja (señal de calidad degradada), distribución de clasificaciones por subpoblación (señal de sesgo), latencia por módulo, y tasa de errores de herramienta.

Capa 4: Gobierno de Cumplimiento

Procesos periódicos de verificación de cumplimiento regulatorio: auditorías del audit trail, revisiones de la documentación técnica requerida por el EU AI Act, reportes de actividad al DPO, y revisiones del Comité de Ética.

6.4 Trazabilidad de Decisiones Agénticas (Audit Trail)

El audit trail de GUARDIAN-Health es una base de datos append-only en PostgreSQL con las siguientes garantías de integridad:

- **Inmutabilidad:** Los registros se firman criptográficamente en el momento de inserción. Cualquier modificación posterior es detectable.
- **Completitud:** Cada nodo del grafo LangGraph genera automáticamente una entrada en el audit trail antes de ejecutarse (pre-action) y después de completarse (post-action). No hay acciones del sistema sin registro.
- **Contexto completo:** Cada entrada del audit trail incluye el estado completo del agente en ese momento, no solo un resumen. Esto permite la reconstrucción exacta de cualquier sesión agéntica.
- **Vinculación de cadena:** Cada entrada referencia la entrada anterior mediante su hash, creando una cadena de evidencias que garantiza que no se han eliminado entradas intermedias.

Esquema de Audit Trail:

```
CREATE TABLE audit_trail (
   id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   sequence_num    BIGINT NOT NULL,  -- Secuencia global e inmutable
   session_id      UUID NOT NULL,
   timestamp       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   module          VARCHAR(20) NOT NULL,  -- VIGIL, ASSESS, RESPOND
   agent_id        VARCHAR(100) NOT NULL,
   action_type     VARCHAR(50) NOT NULL,  -- LLM_CALL, TOOL_USE, HITL_REQ,...
   action_detail   JSONB NOT NULL,        -- Parámetros completos de la acción
   state_snapshot  JSONB NOT NULL,        -- Estado completo del agente
   result          JSONB,                 -- Resultado de la acción
   previous_hash   VARCHAR(64) NOT NULL,  -- Hash SHA-256 del registro anterior
    record_hash     VARCHAR(64) NOT NULL,  -- Hash SHA-256 de este registro
    CONSTRAINT audit_trail_sequence UNIQUE (sequence_num)
);

-- La tabla es de solo inserción; se revoca DELETE y UPDATE al rol de agente

REVOKE DELETE, UPDATE ON audit_trail FROM guardian_agent_role;
```

6.5 Explicabilidad y Transparencia

Cada evaluación producida por el Módulo ASSESS incluye una explicación en lenguaje natural generada por el Agente de Síntesis. Esta explicación sigue una plantilla estructurada que cubre: qué se observó (datos clínicos relevantes), qué sugiere (interpretación del Agente Farmacólogo), qué alternativas se consideraron (interpretación del Agente Clínico), qué requiere el marco regulatorio (evaluación del Agente Regulatorio), y cuál es la recomendación y su nivel de confianza.

La explicación se presenta en la interfaz en dos versiones: una versión técnica para farmacólogos (con todos los detalles del razonamiento) y una versión resumida para médicos prescriptores (enfocada en la acción recomendada y su justificación clínica esencial).

7. Plan de Implementación

7.1 Entorno de Desarrollo y Contenedores

Siguiendo el principio de no instalar software directamente en el sistema operativo anfitrión, el entorno de desarrollo se gestiona completamente mediante contenedores Docker y entornos virtuales Python.

Estructura de Contenedores (docker-compose):

```
services:
  neo4j:
    image: neo4j:5.x-enterprise
    environment:
      NEO4J_PLUGINS: '["apoc"]'
    ports: ["7474:7474", "7687:7687"]

  postgres:
    image: postgres:16
    volumes: ["postgres_data:/var/lib/postgresql/data"]
    ports: ["5432:5432"]

  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333"]

  n8n:
    image: n8nio/n8n:latest
    ports: ["5678:5678"]
    environment:
    N8N_BASIC_AUTH_ACTIVE: "true"

  solace:
    image: solace/solace-pubsub-standard:latest
    ports: ["8080:8080", "55555:55555", "1883:1883"]

  guardian-api:
    build: ./api
    ports: ["8000:8000"]
    depends_on: [postgres, neo4j, qdrant]
```

Guardian Health

Entorno Virtual Python:

Se crea un nuevo entorno virtual `.venv-guardian` específico para este caso de estudio, siguiendo el patrón establecido en el bootcamp. El entorno incluye: `langchain`, `langgraph`, `crewai`, `pyautogen`, `langchain-groq`, `langchain-neo4j`, `langchain-qdrant`, `langchain-pinecone`, `langchain-huggingface`, `fastapi`, `uvicorn`, `solace-pubsub-plus`, `pydantic>=2.0`, `sqlalchemy`, `psycopg2-binary`.

7.2 Estructura del Repositorio

```
guardian-health/
├── langgraph/
├── crewai/
├── README.md
├── ARCHITECTURE.md
├── docker-compose.yml
├── .env.example
├── requirements.txt
├── docs/
│   ├── adrs/
│   └── diagrams/
├── data/
│   ├── synthetic/      (synthetic FHIR patient data)
│   ├── seed/           (initial data loading scripts)
│   └── knowledge/      (clinical guidelines and drug information)
├── src/
│   ├── guardian/
│   │   ├── __init__.py
│   │   ├── config.py
│   │   ├── state.py
│   │   ├── models.py
│   │   └── governance/
│   │       ├── audit.py
│   │       ├── hitl.py
│   │       ├── sanitiser.py
│   │       └── tracer.py
│   ├── modules/
│   │   ├── vigil/
│   │   ├── assess/
│   │   └── respond/
│   ├── tools/
│   ├── data/
│   │   ├── neo4j/
│   │   ├── postgres/
│   │   └── vector/
│   └── api/
└── tests/
```

```
├── unit/
├── integration/
└── governance/
```

7.3 Roadmap de Desarrollo por Fases

- **Fase 0: Fundamentos de Gobierno (Semana 1):** Antes de escribir un solo agente, se implementan las infraestructuras de gobierno: el schema del audit trail en PostgreSQL, el sistema de pseudoanonimización, la configuración de LangSmith, y el estado base del grafo LangGraph con su estructura de trazabilidad. Esto es deliberado: el gobierno primero, los agentes después.

- **Fase 1: Infraestructura de Datos (Semana 1-2):** Carga de datos sintéticos, configuración de Neo4j con el esquema clínico, indexación de documentos en Pinecone/Qdrant, y pipeline de validación de calidad de datos.

- **Fase 2: Módulo VIGIL (Semana 2-3):** Implementación del grafo LangGraph de monitorización con sus herramientas de acceso a datos clínicos y el sistema de publicación de señales via Solace.

- **Fase 3: Módulo ASSESS (Semana 3-4):** Implementación del crew CrewAI de evaluación, el proceso de debate AutoGen para casos ambiguos, y los mecanismos HITL.

- **Fase 4: Módulo RESPOND y API (Semana 4-5):** Implementación del módulo de respuesta, la API FastAPI, el dashboard de gobierno, y los workflows n8n de automatización.

- **Fase 5: Integración y Validación (Semana 5-6):** Pruebas de integración completas, validación del audit trail con escenarios de auditoría, y generación de documentación de conformidad.

8. Apéndices

A. Registro de Decisiones de Arquitectura (ADRs)

ADR-001: LangGraph como Orquestador de Flujo Agéntico

- Estado: Aceptado
- Contexto: El sistema requiere flujo agéntico con estado persistente y capacidad de HITL como interrupción del grafo.
- Decisión: LangGraph sobre alternativas como Prefect o Airflow porque proporciona soporte nativo para agentes LLM con estado, interrupciones para HITL, y checkpointing en PostgreSQL.
- Consecuencias: Curva de aprendizaje de LangGraph. Dependencia del ecosistema LangChain. Ganancia: trazabilidad integrada y HITL como ciudadano de primera clase del flujo.

ADR-002: PostgreSQL para Audit Trail sobre Bases de Datos de Series Temporales

- Estado: Aceptado
- Contexto: El audit trail requiere inmutabilidad, integridad referencial con los datos clínicos, y capacidad de queries complejas para auditoría.
- Decisión: PostgreSQL con tabla append-only sobre InfluxDB o Cassandra porque las queries de auditoría requieren JOINs con datos clínicos que son más naturales en SQL, y PostgreSQL soporta la restricción REVOKE para garantizar append-only a nivel de base de datos.
- Consecuencias: PostgreSQL puede no escalar tan bien como bases de datos de series temporales en volumen extremo. Para la escala actual (red de 3 hospitales), es apropiado.

ADR-003: Pseudoanonimización en Lugar de Anonimización Completa

- Estado: Aceptado
- Contexto: El GDPR permite el uso de datos de salud bajo base legal legítima pero requiere medidas técnicas apropiadas. La anonimización completa destruiría la utilidad del sistema para farmacovigilancia individual.
- Decisión: Pseudoanonimización de dos capas (UUID permanente + ID de sesión temporal) en lugar de anonimización completa, porque la anonimización completa es incompatible con la función de detectar y gestionar eventos adversos en pacientes individuales.
- Consecuencias: Los datos pseudoanonimizados siguen siendo datos personales bajo el GDPR. Requiere controles adicionales (tabla de mapping con acceso muy restringido, auditoría de accesos a la tabla).

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

ADR-004: Groq como Proveedor de LLM Principal

- Estado: Aceptado
- Contexto: El sistema requiere inferencia de LLM de baja latencia para procesamiento en tiempo (casi) real. Los modelos locales son lentos para el hardware disponible.
- Decisión: Groq (llama-3.3-70b-versatile) sobre OpenAI GPT-4 porque la latencia de Groq es significativamente menor, el coste es menor, y la soberanía del modelo (Llama es open-weights) es preferible para un contexto regulado.
- Consecuencias: Los datos enviados a Groq salen del perímetro controlado. Mitigación: política estricta de pseudoanonimización antes de cualquier envío a Groq (PD-01).

ADR-005: Solace Agent Mesh para Mensajería Inter-Módulo

- Estado: Aceptado
- Contexto: Los tres módulos del sistema son semi-independientes y se comunican via eventos asíncronos.
- Decisión: Solace Agent Mesh sobre Kafka o RabbitMQ porque Solace está diseñado específicamente para architecturas de agentes IA con soporte para patrones de mensajería agéntica, y está disponible como Docker image para desarrollo.
- Consecuencias: Solace tiene una curva de aprendizaje mayor que Kafka. La madurez del ecosistema de Solace Agent Mesh para IA agéntica es menor que la de Kafka. Revisable en producción.

ADR-006: Repositorio Independiente para GUARDIAN-Health

- Estado: Accepted.
- Context: Los casos de estudio anteriores (Financial Crime, Tourism) viven bajo `agentic-ai-bootcamp`. GUARDIAN-Health tiene un alcance arquitectónico significativamente mayor (tres módulos, infraestructura completa, documentación regulatoria) que justifica un repositorio propio con su propio ciclo de vida.
- Decision: Repositorio separado `github.com/mgalanme/guardian-health` en lugar de subcarpeta en `agentic-ai-bootcamp`.
- Consecuencias: Los dos repositorios coexisten. El README de `agentic-ai-bootcamp` debe referenciar `guardian-health` como caso de estudio externo. El `.env` y secretos se gestionan de forma completamente independiente.

Gobierno y Responsabilidad por Diseño en IA Agéntica

Guardian Health

B. Glosario

- **Agente:** Entidad de software que usa un LLM para razonar y tomar decisiones, con acceso a herramientas para actuar en su entorno.

- **Audit Trail:** Registro cronológico, inmutable y completo de todas las acciones y decisiones de un sistema, diseñado para soportar auditorías internas y externas.

- **Crew (CrewAI):** Grupo de agentes especializados que colaboran en la resolución de una tarea compleja, con roles definidos y proceso de coordinación.

- **Linaje de Datos:** Trazabilidad del origen, transformaciones y uso de un dato a lo largo de su ciclo de vida.

- **Data Steward:** Persona responsable de la calidad, uso correcto y cumplimiento de políticas para un dominio específico de datos.

- **E2B R3:** Estándar internacional (ICH E2B) para el intercambio electrónico de información de seguridad individual de casos (ICSR) en farmacovigilancia.

- **Event Adverse (RAM):** Reacción Adversa a Medicamento. Respuesta no deseada y perjudicial a un medicamento.

- **FHIR R4:** Fast Healthcare Interoperability Resources. Estándar internacional para el intercambio de datos de salud electrónicos.

- **Graph RAG:** Retrieval-Augmented Generation sobre una base de datos de grafos, donde la recuperación aprovecha las relaciones del grafo.

- **HITL (Human-in-the-Loop):** Patrón de diseño donde un humano participa en el ciclo de decisión de un sistema automatizado, especialmente para decisiones de alto riesgo.

- **LangGraph:** Framework de LangChain para construir flujos agénticos con estado como grafos dirigidos con soporte para checkpointing y HITL.

- **Medallion Architecture:** Patrón de organización de pipelines de datos en capas Bronze (bruto), Silver (limpio), Gold (preparado para consumo).

- **ReAct (Reasoning + Acting):** Patrón de agente que alterna entre razonamiento (pensar sobre la situación) y actuación (invocar herramientas).

- **SAR (Suspicious Activity Report):** Informe de actividad sospechosa. En farmacovigilancia, el documento de notificación de sospecha de RAM.

- **Solace Agent Mesh:** Infraestructura de mensajería orientada a eventos diseñada para arquitecturas de agentes IA.

- **TOGAF 10:** The Open Group Architecture Framework versión 10. Framework de arquitectura empresarial.

- **DAMA DMBOK2:** Data Management Body of Knowledge versión 2. Guía de referencia para gestión y gobierno del dato.

C. Referencias

Frameworks y Estándares:

- The Open Group: TOGAF® Standard, Version 10 (2022)
- DAMA International: DAMA-DMBOK: Data Management Body of Knowledge, 2nd Edition
- ICH: Guideline E2B(R3) - Electronic Transmission of Individual Case Safety Reports (2013)
- European Medicines Agency: Good Pharmacovigilance Practices (GVP) Modules
- Real Decreto 577/2013, de 26 de julio, de Farmacovigilancia de Medicamentos de Uso Humano.

Regulación:

- Reglamento (UE) 2024/1689 (EU AI Act)
- Reglamento (UE) 2016/679 (GDPR)
- Ley Orgánica 3/2018 (LOPDGDD)
- Reglamento (UE) No 1235/2010 sobre Farmacovigilancia.

Tecnologías:

- LangGraph Documentation: https://langchain-ai.github.io/langgraph/
- CrewAI Documentation: https://docs.crewai.com/
- Microsoft AutoGen: https://microsoft.github.io/autogen/
- Neo4j Graph Data Science: https://neo4j.com/docs/graph-data-science/
- Solace Agent Mesh: https://solace.com/solutions/artificial-intelligence/
- Groq Documentation: https://console.groq.com/docs/